Part 1: Basic SQL Operations and JOIN Queries

1) **Basic Selection: Retrieve the titles and publication years of all books published after 2000, ordered by publication year (newest first).**

```
SELECT title, publication_year
FROM books
WHERE publication_year > 2000
ORDER BY publication_year DESC;
```

2) **Filtering: Find all books with more than 5 copies owned in the fiction genre (genre_id = 1).**

```
SELECT *
FROM books
WHERE copies_owned > 5
  AND genre_id = 1;
```

3) **Pattern Matching: List all books whose titles contain the word "History".**

```
SELECT *
FROM books
WHERE title LIKE '%History%';
```

4) **JOIN Operations: Display loan information (loan_id, checkout_date, due_date) along with patron details (first_name, last_name, email) for all loans made in January 2023.**

```
SELECT
l. loan_id,
l. checkout_date, l.due_date,
p. first_name,
p. last_name,
p. email
FROM loans AS 1
INNER JOIN patrons AS p
ON l. patron_id = p. patron_id
WHERE 1. checkout_date >= '2023-01-01' AND l. checkout_date < '2023-02-01';
```

5) Multi-table JOIN: Show book details (title, author's full name, genre_name) for each loan, along with the checkout_date and due_date.

```
SELECT
b.title,
CONCAT (a first_name,' ', a.last_name) AS author_full_name,
g- genre_name,
l. checkout_date,
l. due_date
FROM loans AS 1
INNER JOIN books AS b
ON 1. book_id = b. book_id
INNER JOIN authors AS a
ON b.author_id = a.author_id
INNER JOIN genres AS g
ON b-genre_id = g.genre_id;
```

6) Self JOIN: Find pairs of patrons who live in the same city. Show both patrons' names and their city.

```
SELECT
p1.first_name AS patron1_first_name,
p1. Last_name AS patron1_last_name,
p2. first_name AS patron2_first_name,
p2. last_name AS patron2_last_name,
p1. city
FROM patrons AS p1
INNER JOIN patrons AS p2
ON p1.city = p2. city
AND p1.patron_id < p2.patron_id;
```

7) Multi-table JOIN with filtering: Find all fiction books (genre_id = 1) that have been borrowed, along with the patron name and the branch where they were borrowed from.

```
SELECT
b.title,
CONCAT (p.first_name, ' ', p.last_name) AS patron_name,
br. branch_name
FROM books AS b
INNER JOIN loans AS l
ON b.book_id = l. book_id
INNER JOIN patrons AS p
ON l. patron_id = p.patron_id
INNER JOIN branches AS br
ON l. branch_id = br.branch_id
WHERE b.genre_id = 1;
```

Part 2: Aggregation and GROUP BY Operations

8. COUNT aggregation: Count the number of books in each genre category.

```
SELECT
g.genre_name,
COUNT (b.book_id) AS number_of_books
FROM genres AS g
LEFT JOIN books AS b
ON g.genre_id = b.genre_id
GROUP BY g.genre_id, g.genre_name
ORDER BY number_of_books DESC;
```

9. Multiple aggregations: Calculate the average, minimum, and maximum loan duration (days between checkout and return) for each library branch. Include only returned books.

```
SELECT
br.branch_name,
AVG (DATEDIFF (l.return_date, l.checkout_date) ) AS avg_loan_days,
MIN (DATEDIFF (l.return_date, l.checkout_date)) AS min_loan_days,
MAX (DATEDIFF (l.return_date, l.checkout_date)) AS max_loan_days
FROM loans AS l
INNER JOIN branches AS br
ON l.branch_id = br.branch_id
WHERE l.return_date IS NOT NULL
GROUP BY br.branch_id, br.branch_name
ORDER BY br.branch_name;
```

10. Conditional aggregation: Find patrons with overdue books (due_date < CURRENT_DATE and return_date = ' '), along with the count of overdue books they have.

```
SELECT
p. first_name,
p. last_name,
p.email,
COUNT (*) AS overdue_book_count
FROM patrons AS p
INNER JOIN loans AS l
ON p.patron_id = l.patron_id
WHERE l.due_date < CURRENT_DATE
AND (l.return_date IS NULL OR TRIM(1. return_date) = '')
GROUP BY p.patron_id,p.first_name,p.last_name,p.email
HAVING COUNT (*) > 0
ORDER BY overdue_book_count DESC;
```

11. Time-based analysis: Analyze monthly borrowing trends. Show the year, month, number of loans, and number of unique patrons for each month.

```
SELECT
YEAR (l.checkout_date) AS loan_year,
MONTH (l.checkout_date) AS loan_month,
COUNT (*) AS number_of_loans,
COUNT (DISTINCT l.patron_id) AS unique_patrons
FROM loans AS l
GROUP BY
YEAR (l.checkout_date),
MONTH(l.checkout_date)
ORDER BY
loan_year,
loan_month;
```

Discussion Questions

1) **In our library database, we track which branch a book was borrowed from, but books can exist at multiple branches. How would you modify the schema to track the actual inventory at each branch?**

Right now, the model only records which branch a loan occurred at and in order to properly track inventory, there should be a copies table. The loans table should also have a copy_id. By adding this table, it allows us to answer the following questions: how many copies of a title are currently available at a specific branch and which branches have no copies of a popular title.

2) **Based on the provided data model, what business questions could library administrators answer using SQL queries that we haven't covered in our exercise?**

The business questions that could be answered is as follows:

- Which genres or authors generate the most late returns?
- Which genres are growing in popularity?
- Who are the most active patrons?
- Which books are most popular?
- What is the average loan duration by branch or genre?
- Which branch circulates the most books?

3) **How would you extend this schema to track additional patron interactions, such as reserved books, late fees, or participation in library programs?**

I would extend this schema by introducing reservations/holds, late fees/fines, and library programs/events

4) **For tasks 1-3, how could you combine them into a single, more complex query that finds recent history books with multiple copies?**

A potential query could be as follows:

```
SELECT
   b.BookID,
   b.Title,
   COUNT(DISTINCT bi.BranchID) AS BranchCount,
   SUM(bi.AvailableCopies) AS TotalAvailableCopies,
   COUNT(DISTINCT r.ReservationID) AS ReservationCount
```

```
FROM Books b
JOIN BookInventory bi
   ON b.BookID = bi.BookID
LEFT JOIN Reservations r
   ON b.BookID = r.BookID
WHERE b.Genre = 'History'
  AND b.PublishDate >= '2023-01-01'
GROUP BY b.BookID, b.Title
HAVING COUNT(DISTINCT bi.BranchID) > 1
   AND SUM(bi.AvailableCopies) > 1
ORDER BY ReservationCount DESC;
```

5) What performance considerations should be kept in mind when running complex joins and aggregations on large library datasets?

Some performance considerations that should be kept in mind while running complex joins/aggregations is indexing, composite indexing, only retrieving the columns that are needed, date-range filtering, aggregation strategy which includes filtering before aggregation and joins such as inner or left.